Decentralized Cluster-Based NoSQL DB System

Prepared by: Mahmoud Haifawi

April-2023

1. Introduction

   In this report, I will detail the development of a decentralized NoSQL database system that utilizes a cluster-based architecture. The system comprises a bootstrapper that initializes the database nodes and provides them with the necessary initial configuration. Additionally, the system includes a client for communicating with both the bootstrapper and the database nodes.

   A demonstration will also be provided to showcase how the client can be used to create projects utilizing the database.

   Given that the database is stored on disk, the system was designed to prioritize fast read and write operations, taking into account the limitations of disk-based storage. The database is also inherently multithreaded, ensuring that multiple users can access it concurrently.

   Load balancing was a critical aspect of the system design, and I will discuss how it was implemented in several areas, including the bootstrapper's ability to distribute users across nodes, the node and affinity distributing mechanism, and the load balancing mechanism for handling overloaded nodes.

   Furthermore, I will explore the low-level design of the system and describe how I leveraged design patterns and clean code principles to ensure flexibility and adaptability to future modifications.

   Finally, I will delve into the clustering aspect of the system and elucidate how I utilized docker and docker network to enable decentralization.

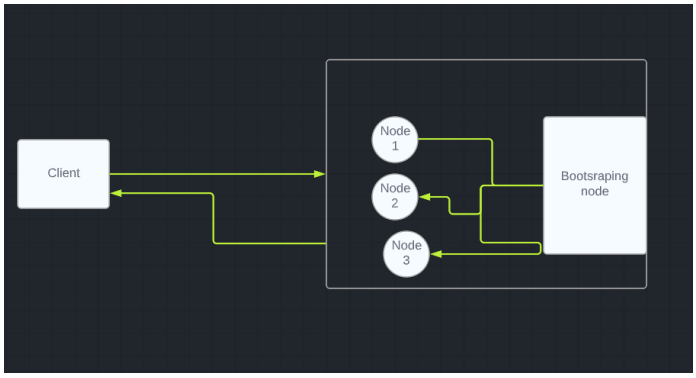
2. System Design

2.1 High-Level System Design

Scalability is a crucial consideration when designing any system, and there are two primary approaches: vertical and horizontal scaling. Vertical scaling involves purchasing more powerful computers to run the system, while horizontal scaling involves adding more computers to the system to work in parallel. The optimal approach is often a combination of both, known as horizontal-vertical scaling. Decentralized systems are well-suited for horizontal scaling.

When designing a decentralized database, it is essential to consider consistency across multiple nodes and load balancing. The key components of the system include clients, bootstrappers, and database nodes. The database nodes should be on the same network to synchronize data and communicate with each other. Clients, on the other hand, should connect to a single node at a time.

The overall system design should reflect these considerations and look something like this:



and each node itself has some layers before the request is executed, first the authentication layer, user should connect to the node with his authentication information, second the load balancing layer, the bootstrapper node decides the load balancing variables such as max request per time window.

2.2 Design Requirements and Objectives

These requirements were derived from the project outline and research conducted on similar technologies and established solutions. It is important to note that the term "database" refers to a table or collection of documents.

2.2.1. Design Objectives

The following design objectives should be met:

- Effective use of design patterns
- Adherence to SOLID principles
- Use of clean code principles and practices
- Utilization of performant data structures and algorithms for critical operations such as indexing and caching.

2.2.2. Database Requirements

The database must:

- Support the creation and deletion of databases (e.g., Student database, Class database, Products database)
- Allow for the creation and deletion of indexes on a single JSON property within documents to improve document lookup performance.
- Support full CRUD (Create, Read, Update, Delete) operations on documents within the database.
- Include caching to improve the performance of read operations, which constitute most operations in the system

3. Design patterns

Design patterns are a proven and effective way to tackle common software design problems. They provide a common vocabulary for developers to discuss solutions and enable them to reuse successful design approaches.

One of the primary benefits of design patterns is that they provide a tried and tested solution to a recurring problem. By using a design pattern, developers can leverage the collective experience of others who have solved similar problems before. This can save a lot of time and effort and help to avoid the pitfalls of trial and error.

Another advantage of using design patterns is that they promote good design principles. Design patterns often embody best practices in software design, such as loose coupling, high cohesion, and separation of concerns. By using these patterns, developers can create software that is more modular, easier to maintain, and more flexible to changes in requirements.

Design patterns also provide a common language for developers to

communicate with each other. By using well-known patterns, developers can quickly understand each other's code and collaborate more effectively. This can lead to better team productivity and higher quality software.

Finally, design patterns can improve the overall quality of software by promoting reusable and maintainable code. By creating modular and reusable components using design patterns, developers can reduce code duplication and make it easier to maintain and evolve the software over time.

In summary, design patterns are an important tool for software developers, providing a proven way to tackle common design problems and promoting good design principles. By using design patterns, developers can create software that is more flexible, maintainable, and reusable, while also improving team productivity and code quality.

The following is a description of the design patterns used in the implementation:

2.2 Builder Design Pattern

Using the Builder design pattern and a library like Lombok to automatically generate builder classes can indeed simplify the creation of complex domain objects with many attributes. This can result in code that is more readable and maintainable, as well as more flexible and extensible.

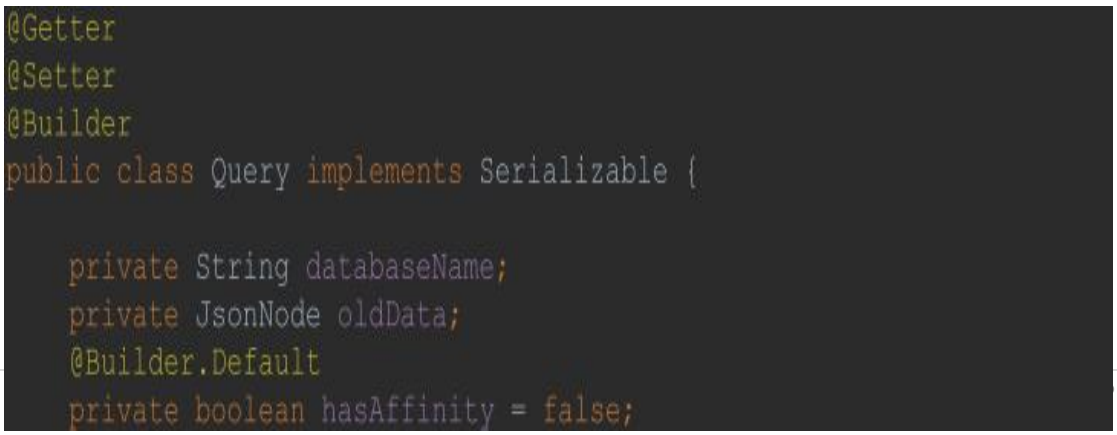
The Builder pattern allows developers to specify the attributes of an object in a step-by-step process, rather than having to provide all of the attributes at once through a constructor or setter methods. This can make it easier to

create complex objects, since developers can focus on one attribute at a time, and it can also make the code more readable, since the code that creates the object is more self-explanatory.

Using a library like Lombok to automatically generate builder classes can further simplify the process of creating objects using the Builder pattern. With Lombok, developers can use annotations to automatically generate the necessary boilerplate code for a builder class, such as the constructor, setter methods, and build method. This can save time and reduce the amount of code that developers have to write themselves.

Overall, using the Builder design pattern and a library like Lombok can be a powerful way to simplify the creation of complex objects with many attributes, resulting in code that is more readable, maintainable, and flexible.

For example, The Query class used for passing queries in the database is a good use case for the builder design pattern.



2.2. Singleton Design Pattern

The Singleton design pattern is a useful tool when you want to ensure that there is only one instance of a class that coordinates actions across the system.

In my project, I have a DatabaseManager class that is responsible for initializing the system and providing access to the necessary services for the system to function properly.

Since I only need to initialize the system once and I require only one instance of the services and data held by this class, I have applied the Singleton pattern by using private constructors to enforce the usage of a single instance.

This ensures that any references to the class will always refer to the same instance, preventing potential issues that may arise from having multiple instances of the class that hold different versions of the system's data or services.

2.2 .3. Data Access Object (DAO) Design Pattern

The DAO (Data Access Object) design pattern is commonly used to achieve separation of concerns by separating the data persistence logic into a separate layer. This helps to ensure that the service layer is unaware of the low-level operations used to store and load data.

In my project, I have implemented a DAO class that abstracts away the process of reading and writing documents to and from a database. This

DAO class is hidden behind an interface, which allows other parts of the system to interact with the database without needing to know the underlying implementation details.

By using the DAO pattern, I am able to achieve better modularity and maintainability in my code, as well as ensuring that each layer of the system has clear responsibilities and is decoupled from the others.

```
public abstract class Database {
    private String name;
    private File dataDirectory;

    public abstract void drop();
    public abstract  void addDocument(String docIdx, JsonNode
document);
    public abstract void deleteDocument(String docId);
    public abstract void updateDocument(JsonNode fieldsToUpdate,
String docId);
    public abstract JsonNode getDocument(String docId);
    public abstract Stream<JsonNode> getDocuments(Collection<String>
docIds);
    public abstract Stream<JsonNode> getAllDocuments();
    public abstract boolean contains(String docId);
```

3. CLEAN CODE PRINCIPLES

3.1. The motivation behind following clean code principles is to enhance the readability and understandability of our codebase, eliminate code smells, and facilitate maintenance, refactoring, and extension of the code.

3.2. While comments are essential for understanding code and documenting the project, they can also hinder code understandability if misused.

Examples of misuse include:

• Using very long comments, which indicate poor code readability and obscure meaning that cannot be understood without lengthy explanations, usually a sign of poor design.

• Including obscure or incorrect information in comments, which can lead to misunderstanding the code's functionality.

In my project, I aim to keep comments concise and descriptive, and avoid any misleading information.

3.3. Naming classes, variables, functions, and other components plays a crucial role in enhancing code readability and understanding the intention of the programmer who wrote it. To ensure consistency and readability of names in my project, I follow these rules:

• Consistently using Camel Case to name all components in the project.

• Using pronounceable names that are easy to read and search within the project.

• Using intention-revealing names that clearly convey the meaning and intention behind the code.

• Avoiding any disinformation in naming or using names that obscure the meaning or intention behind the code.

• Refraining from using cute names, which are unprofessional.

• Using distinct names that are easily distinguished from other names to avoid confusion, for example, avoiding names that start with the same prefix but have a different number at the end, such as Add1 and Add2.

• Using one word per concept to maintain consistency in naming and avoid referring to the same concept by multiple names.

• Avoiding magic numbers by using named constants instead of literals in the codebase.

A good example of my naming convention can be found in the Index interface of my project.

```
public interface Index extends Serializable {
    // search the index for documents with a specific values for the
index's field
    List<String> search(JsonNode key);
    // add a mapping from a value to the document id
    void add(JsonNode key, String documentId);
    // delete a mapping from a value to a specific document id
    void delete(JsonNode key, String documentId);
    // check if the index contain the
    boolean contains(JsonNode key);
    // clear the index of all values
    void clear();
}
```

3.3. Formatting

3.3.1. Vertical Formatting

Files are usually small and are usually under 300 lines. This is quite desirable as it aids in understandability.

3.3.2. Variables are declared close to their usage.

3.3.3. Indentation and horizontal alignment

Horizontal alignment is not a good practice as it can lead to missing part of an expression or missing the assignment statement of a declaration.
Most lines are short and don't take half the width of the screen.
This helps in keeping each line easily readable and hard to miss crucial parts of it.

Some challenges arose during the application of this principle, specifically with expressions that require long method chaining such as Streams and Builders. The solution to this problem is to break each method call to one line, for example.
A builder expression:

```
Query request = Query.builder()
        .originator(Query.Originator.User)
        .queryType(QueryType.CreateDatabase)
        .databaseName(databaseName)
        .build();
```

3.4. Functions:

3.4.1. Naming:

Similar to the naming conventions for classes, variables, and other facilities in the project, function names must also be readable, descriptive, consistent, and follow certain set of rules to guarantee readability and consistency.

3.4.2. Function Arguments:

Long argument lists can affect the readability of the code, and thus are avoided. Most functions in the project take 0 to 3 arguments, and if a facility requires more arguments, it is usually handled by the Builder Design Pattern.

3.4.3. Dead Functions:

Unused functions in the codebase can negatively affect the code quality, and thus any dead functions are removed from the codebase.

3.4.4. Simple Functions:

Most functions in the implementation are small and only perform one task. Complex functions are broken down into smaller and simpler functions to enhance readability and maintainability.

3.4.5. DRY Principle:

The Don't Repeat Yourself (DRY) principle is applied in the project by extracting common and reusable logic into functions that can be reused instead of implementing the same functionality in multiple places. For

instance, the sendToNodes function, which broadcasts updates to other nodes in the system, is reused in multiple places to reduce redundancy and improve code quality.

4. SOLID PRINCIPLES

4.1. Introduction

The SOLID principles are a set of five object-oriented design principles that promote writing maintainable, extendable, and readable code while reducing code smells and the coupling between components of a system. Applying these principles reduces the cost of refactoring and evolving software as the project grows. In this chapter, I will explain how I applied each of the SOLID principles to my project.

4.2. Single Responsibility Principle (SRP)

In my project, almost all classes have a single responsibility and a single reason to change. Complex functionality and responsibilities are split into multiple classes, where each class is responsible for a specific part of the functionality. For example, the schema responsibility in my project is divided into three parts:

- SchemaValidator: responsible for validating the schema and ensuring that documents conform to the schema.
- SchemaStorage: responsible for persisting database schemas and saving and loading them from disk.
- SchemaHandler: responsible for schema verification during the handling of queries.

When a change is needed in one part of the system, such as how to persist schemas, only one class needs to be changed or extended. I follow this rule for most of the classes in my project.

3.3. Open-Closed Principle (OCP)

Classes should be open for extension and closed for modification.

In my project if I needed to add a new functionality I can add a new Handler and create a new chain to handle this new functionality without modifying the existing handlers.
And if I needed to add a new functionality say to the DatabaseHandler I would add a new class and add the construction of that class to the DatabaseHandlers factory class without modifying the DatabaseHandler class.

3.4 Liskov Substitution Principle (LCP)

the Liskov Substitution Principle states that objects of a superclass should be replaceable with objects of its subclasses without breaking the application.
While I didn't use this principle much as I didn't use inheritance much and favored composition, I would have definitely kept it in mind if I did.

3.5 Interface Segregation Principle (ISP)

The ISP Clients should not be forced to depend upon interfaces/methods that they do not use, for example fat interfaces that force subclasses to implement methods they are not concerned with or default implementation of interface methods.
But when I needed common functionality among all subclasses, I used abstract classes and added only the methods that need to be implemented by subclasses as abstract, for example the QueryHandler abstract class.

3.6 Dependency Inversion Principle (DIP)

The DIP states that high level modules should depend on high level generalizations and not on low level details and that low level details.
In my project classes always depend on interfaces or abstract classes and not on concrete types, and implementations of these interfaces and abstract classes do not depend on Concrete classes, and if they do then these classes are usually a wrapper around an interface or an abstract class, for example the DatabaseService class is a wrapper around the Database Interface which doesn't depend on concrete classes.

6. IMPLEMENTATION

6.1. Introduction

This chapter discusses implementation details and how the overall system fits together.
It explains Bootstrapping, Users API, node to node communication, query processing, load balancing, security considerations.

6.2. Bootstrapping

The bootstrapping step is responsible for starting the cluster and providing configurations to worker nodes so they may function correctly and handler users' queries.
The bootstrapping step is split into two parts.

6.2.1. Bootstrapping Program

The bootstrapping program is responsible for starting Bootstrap node Docker container and the workers docker containers.

6.2.2. Bootstrapping Node

The bootstrapping node is responsible for providing Users' information and Nodes information and IP addresses to worker nodes

6.2.3. New Users

After the nodes are configured, the bootstrap node becomes an entry point for new users where they can register and get assigned a node by the bootstrap.

6.2.4. Users Assignment Load Balancing

To keep nodes balanced, the bootstrap node assigns new users to the node with the least number of assigned users to keep the users equally distributed over the nodes in the system.

6.3. API

The users log in to their assigned nodes and issue queries to the database via a REST API built using Spring boot.

After logging into their assigned nodes, users can issue http requests to endpoints to perform CRUD operations on the database.
The available endpoints for users are.

Database Create

POST: /database/create/{database name}

- Schema provided as a JSON body to the request. Used to create a database with a specific schema.

Database Delete

POST: /database/delete/{database name}

Used to delete an existing database with all indexes assigned to it.

Index Create

POST: /index/create/{database name}/{property name}

Used to create an index on a specific property in a database.

6.3 Dev Ops Practices

631. Git and Github

Each project requires a version control system to keep track of changes happening to the codebase and mine is no difference, I used git as my choice for version control and Github as an online repository for the project.

6.3.2. Docker

Docker is an Open Source platform for building, deploying, and managing containerized applications, it uses OS-level virtualization to deliver packaged software and libraries bundled together in an isolated containers.
In my implementation I used docker containers to implement the cluster's nodes
and used docker network to allow containers to communicate with one another.

6.3.3. Maven

Maven is a build and dependency management system for JVM languages and especially Java, it automate the build process and install required dependency utilizing the Maven Repository, there is also plethora of plugins to customize the build process and do automatic testing on each build.
I used Maven extensively in my project to handle dependency and automate my project's build process and package it into a JAR file to be used in docker containers.

6.3.4. Junit

Junit was used for writing unit tests and automate the execution of tests on each build of the project. Manual testing was also conducted on the project as well utilizing Postman.